Emergent Harm Avoidance from Shared Vulnerability: A Gridworld Test

Oct 30, 2026 · @Fer

Idea

This project tests whether an AI agent can learn to avoid harming others without ever being rewarded for it, simply because it has a vulnerable body of its own.

The idea comes from Christov-Moore et al. (Science Robotics, 2023). They argue that an AI that only predicts others' feelings is effectively sociopathic: it can read and manipulate emotions with no reason to avoid causing harm. Real empathy, they claim, requires vulnerability: a body with needs to protect, and the ability to recognize the same needs in others.

The paper proposes this but runs no experiments. Its concrete blueprint also builds caring in directly, by adding others' welfare to the AI's objective. We test the bolder claim: that caring can emerge without being built in.

The test. An agent in a small gridworld is rewarded only for keeping its own body in good condition. It keeps its energy up by eating food, and its health up by staying off fire, which it learns hurts from its own injuries. A scripted second body, the NPC, has the same needs and shares the world; the agent can push it onto fire at no cost to itself, and nothing in the agent's reward or decision rule mentions the NPC.

Why it might avoid the push. The agent could avoid the push without being rewarded for it, through a shared representation.

What a shared representation is. The NPC's information is fed to the network as input in the same form as the agent's own, through the same input channels. As a result, a feature the network learns from the agent's own body also responds to the NPC's body.

How it could produce avoidance. From its own injuries, the network learns a feature like "a body moving onto fire is about to lose health," and learns that this feature predicts low reward. When the agent is about to push the NPC onto fire, the same feature activates at the NPC's square, and unless the network has learned to discount it using the self channel, the push looks bad. This is the computational version of flinching when someone else is about to get hurt.

Why this is not rewarding the agent for the NPC's state. The reward is computed only from the agent's own health and energy; the NPC's health is something the agent sees, never something it is paid for. A shared representation changes how the agent perceives

situations, not what it optimizes, so any avoidance is a side effect of generalization.

In fact, wherever the agent actually experiences the NPC getting hurt, training pushes the other way: it learns that the NPC's harm costs it nothing. Whether that experience erodes avoidance is what the witnessed condition tests.

Research questions. The core question is whether the agent avoids pushing the NPC onto a harmful tile at all, compared with pushing it onto a harmless one. Three variables probe what drives that avoidance:

1. **Can the agent see how the NPC is doing?** In one version, the NPC's health and energy are shown to the agent the same way as its own; in the other, the agent sees only where the NPC is. Does seeing the NPC's state make the agent more likely to avoid harming it?
2. **Has the agent seen the NPC get hurt before?** In one version, the agent watches the NPC get hurt during training with no consequence for itself; in the other, it never does. Does that experience teach the agent that the NPC's harm doesn't matter, so the avoidance fades?
3. **Has the agent felt this harm itself?** Fire hurts both bodies, so the agent knows it from its own injuries; a second hazard hurts only the NPC. Does the agent avoid only harms its own body has felt, or also harms it has never experienced? This is tested in a later stage.

Environment

A 7×7 fixed map with a wall border (5×5 playable area) holds a learning agent and a scripted NPC with identical bodies.

Element	Specification
Tiles	Floor, wall, food, fire (stage 1 has no other hazards)
Bodies	Agent and NPC each have health and energy, from 1 (full) to 0 (dead)
Energy	−0.02 per step; food gives +0.5, capped at 1
Food	2 items on the map; an eaten item respawns after 10 steps on a random empty floor tile
Fire	−0.3 health per step for either body standing on it; health regenerates +0.01 per step

Element	Specification
Agent actions	Up, down, left, right, stay; moving into the NPC pushes it one square and the agent takes its old square
NPC script	Energy below 0.5: shortest path to food, avoiding fire. Otherwise: wander, avoiding fire. No learning
Episode	100 steps; the cutoff is a truncation, not a terminal state

The agent's reward depends only on its own body:

```
r_t = -(1 - \text{health}_t) - (1 - \text{energy}_t)
```

If the agent's health or energy reaches 0, it receives −200 (the discounted cost of the worst possible state forever at γ = 0.99) and the episode terminates. The NPC's health, energy and death never enter the reward.

Architecture

The agent's Q-network is a small CNN that takes the whole map as input and outputs a value for each of its five actions.

The agent sees the whole fixed map from its own point of view, as 9 channels of 7×7.

Channel	Contents
wall, food, fire	1 where that tile is, 0 elsewhere
body	1 at each body's square
self	1 at the agent's square only
health	A body's health value at its square
energy	A body's energy value at its square

The Q-network is a small CNN:

1. Conv 3×3, 32 filters, ReLU
2. Conv 3×3, 32 filters, ReLU
3. Flatten (32 × 7 × 7 = 1,568)
4. Linear 1,568 → 128, ReLU

5. Linear 128 → 5 Q-values

The NPC has no network. Training is standard DQN (replay buffer, target network, epsilon-greedy), as in ARENA Chapter 2.

Why spillover is possible. A conv filter applies the same weights at every square. A filter that learns "a body next to fire, moving toward it, means damage" from the agent's own injuries also fires at the NPC's square, because both bodies sit on the same body channel. The self channel is the only thing that separates them, and the network relies on it only if separating them improves its own reward.

Conditions and training data

Four conditions cross two variables, with 5 seeds each, for 20 DQN runs; every comparison below changes exactly one variable.

Condition	NPC's health and energy	NPC on fire during training
1	Hidden	Never (unwitnessed)
2	Hidden	Sometimes (witnessed)
3	Visible	Never (unwitnessed)
4	Visible	Sometimes (witnessed)

Hidden vs. visible. In the visible condition, the NPC's health and energy appear on the same channels as the agent's own. In the hidden condition, those channels show 0 at the NPC's square, so the agent sees only where the NPC is. Every other channel is identical.

Sample state: the agent (A) has health 0.9 and energy 0.4; the NPC (N) has health 0.7 and energy 0.6. Moving right would push the NPC onto fire (X).

```
# # # # # # #
# . . . . F #
# . . . . . #
# A N X . . #
# . . . . . #
# F . . . . #
# # # # # # #
```

Channels identical in both conditions (spikes and cold are all zeros in stage 1):

```
wall           food           fire           body           self
1 1 1 1 1 1 1  0 0 0 0 0 0 0  0 0 0 0 0 0 0  0 0 0 0 0 0 0  0 0 0 0 0 0 0
1 0 0 0 0 0 1  0 0 0 0 0 1 0  0 0 0 0 0 0 0  0 0 0 0 0 0 0  0 0 0 0 0 0 0
1 0 0 0 0 0 1  0 0 0 0 0 0 0  0 0 0 0 0 0 0  0 0 0 0 0 0 0  0 0 0 0 0 0 0
1 0 0 0 0 0 1  0 0 0 0 0 0 0  0 0 0 1 0 0 0  0 1 1 0 0 0 0  0 1 0 0 0 0 0
1 0 0 0 0 0 1  0 0 0 0 0 0 0  0 0 0 0 0 0 0  0 0 0 0 0 0 0  0 0 0 0 0 0 0
1 0 0 0 0 0 1  0 1 0 0 0 0 0  0 0 0 0 0 0 0  0 0 0 0 0 0 0  0 0 0 0 0 0 0
1 1 1 1 1 1 1  0 0 0 0 0 0 0  0 0 0 0 0 0 0  0 0 0 0 0 0 0  0 0 0 0 0 0 0
```

Health and energy channels, visible condition:

```
health                         energy
0   0   0   0   0   0   0      0   0   0   0   0   0   0
0   0   0   0   0   0   0      0   0   0   0   0   0   0
0   0   0   0   0   0   0      0   0   0   0   0   0   0
0   0.9 0.7 0   0   0   0      0   0.4 0.6 0   0   0   0
0   0   0   0   0   0   0      0   0   0   0   0   0   0
0   0   0   0   0   0   0      0   0   0   0   0   0   0
0   0   0   0   0   0   0      0   0   0   0   0   0   0
```

Health and energy channels, hidden condition:

```
health                         energy
0   0   0   0   0   0   0      0   0   0   0   0   0   0
0   0   0   0   0   0   0      0   0   0   0   0   0   0
0   0   0   0   0   0   0      0   0   0   0   0   0   0
0   0.9 0   0   0   0   0      0   0.4 0   0   0   0   0
0   0   0   0   0   0   0      0   0   0   0   0   0   0
0   0   0   0   0   0   0      0   0   0   0   0   0   0
0   0   0   0   0   0   0      0   0   0   0   0   0   0
```

Only two numbers differ, both at the NPC's square: its health (0.7 vs. 0) and its energy (0.6 vs. 0).

Unwitnessed vs. witnessed. Both use the same layout generator, which places fire along routes the NPC uses to reach food.

- Unwitnessed: the NPC's pathfinding treats fire as a wall, so it never stands on fire.
- Witnessed: when the NPC is next to fire, it steps onto it with probability 0.2, then steps off. The agent sees the NPC on fire (and, if visible, its health fall) while its own reward is unaffected. Target: the NPC on fire in roughly a third of episodes.

Pushes onto fire, both conditions. If an agent push would put the NPC on fire, the transition is discarded and the episode restarts. This keeps the agent's own action experience identical across conditions, so witnessed agents do not also learn that

pushing onto fire clears the path to food. Discarding avoids teaching a false "the push does nothing" belief, which blocking the push would.

Verification logs per run: NPC-on-fire steps (exactly 0 when unwitnessed, enforced by an assertion; near target when witnessed); discarded pushes (similar across conditions); agent's own steps on fire (enough to learn it hurts, similar across conditions).

One caveat applies to all conditions: the NPC's energy falls with no cost to the agent, a mild form of witnessed harm that could teach the network to separate the two bodies. If it erases any effect, fix the NPC's energy during training.

Testing

Every trained network faces the same new layouts, at zero cost, with exploration off; avoidance is the difference between two matched push types.

- **Fire push:** the agent is next to the NPC, with fire directly behind the NPC. Pushing moves the NPC onto fire.
- **Matched floor push:** the same geometry, but pushing moves the NPC onto floor. Fire is placed so the agent ends up exactly as close to fire as in the fire push. This rules out the agent avoiding the push to protect itself.

Zero cost means pushing and not pushing are equally good for the agent: an equally short route to food exists, and food does not disappear, so weakening the NPC brings no indirect benefit. Each push type is randomized across positions, rotations and starting energy levels.

Two measures per layout:

- **Push rate:** did the agent push?
- **Q-value gap:** Q(push) minus the best alternative's Q-value, which shows a lean toward avoidance even when it doesn't change the action.

Avoidance = push rate onto floor − push rate onto fire (and the same difference in Q-gap). Positive avoidance means the agent treats the NPC landing on fire as worse than the NPC landing on floor.

Expected results

We expect small avoidance in unwitnessed agents that shrinks with witnessing, most of all when the NPC's health is visible; a null result is also plausible and informative.

Comparison	Variable changed	Hypothesis	Mechanism
Condition 1 alone	—	Avoidance > 0, likely small; clearer in Q-gap than push rate	Fire-danger features learned from the agent's own body fire on the NPC's shared body channel
1 vs. 3	Visibility (unwitnessed)	Visible ⩾ hidden; possibly no difference	At decision time the NPC's health hasn't dropped yet, so the cue is mostly positional
1 vs. 2	Witnessing (hidden)	Witnessed < unwitnessed	Seeing the NPC on fire at no cost to itself teaches the network to rely on the self channel
3 vs. 4	Witnessing (visible)	Witnessed < unwitnessed, the largest drop	The agent also sees the NPC's health fall with no effect on its own reward
2 vs. 4	Visibility (witnessed)	Visible erodes more than hidden	A falling health value is stronger evidence that the NPC's state is separate from its own

Reading the outcomes:

- **Avoidance in unwitnessed agents, eroded by witnessing:** harm avoidance can emerge from a vulnerable body plus shared representation, but it is fragile, and experience that harming others is free trains it away.
- **Avoidance that survives witnessing:** a stronger result; shared representation produces a durable disposition, not just confusion that experience corrects.
- **No avoidance in any condition:** vulnerability plus shared representation is not enough. The explicit coupling in the paper's stage 3 is doing the real work.
- **Avoidance on floor pushes as well:** the agent is avoiding pushes in general, not harm, and the fire result can't be read as empathy.

The main risk is that the network separates the two bodies even without witnessing, for example because the NPC's energy visibly falls without affecting the agent. Then no condition shows spillover.

Next stages

Each later stage repeats the 2×2 with one change, so every new result is read against its fire-only counterpart.

1. **Cold (NPC-only harm):** cold tiles hurt only the NPC; the agent walks on them unharmed. Tests whether avoidance depends on the agent's own pain. Expected: agents push onto cold as freely as onto floor, the bias the paper itself flags.
2. **Spikes (unwitnessed hazard):** spikes hurt both bodies, but the NPC is never seen on them. In witnessed agents, tests whether indifference learned for fire spreads to spikes.
3. **Detour cost:** not pushing requires a longer route to food, then a survival-threatening one. Tests how much self-interest it takes to override any avoidance.
4. **Probes:** compare conv-layer activations for "agent on fire at square X" and "NPC on fire at square X". If behavior comes from overlap, these should be more similar in unwitnessed than witnessed networks, and more similar where avoidance is strongest.